Bharath

2403A51L17

Batch-51

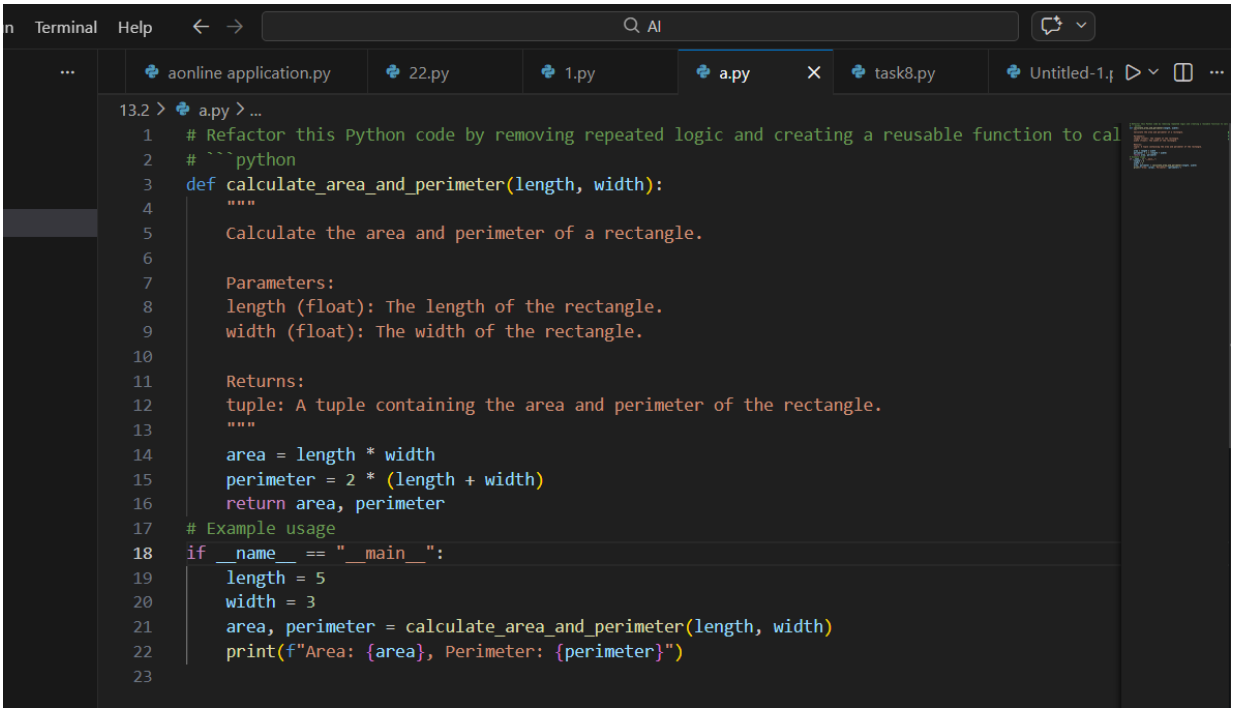
ASSIGNMENT 13.2

Code Refactoring – Improving Legacy Code with AI Suggestions

Task Description #1 (Refactoring – Eliminating Repetitive Logic):

PROMPT: Refactor this Python code by removing repeated logic and creating a reusable function to calculate area and perimeter. Add a docstring and keep the same output.

CODE:



OUTPUT:

Explanation :

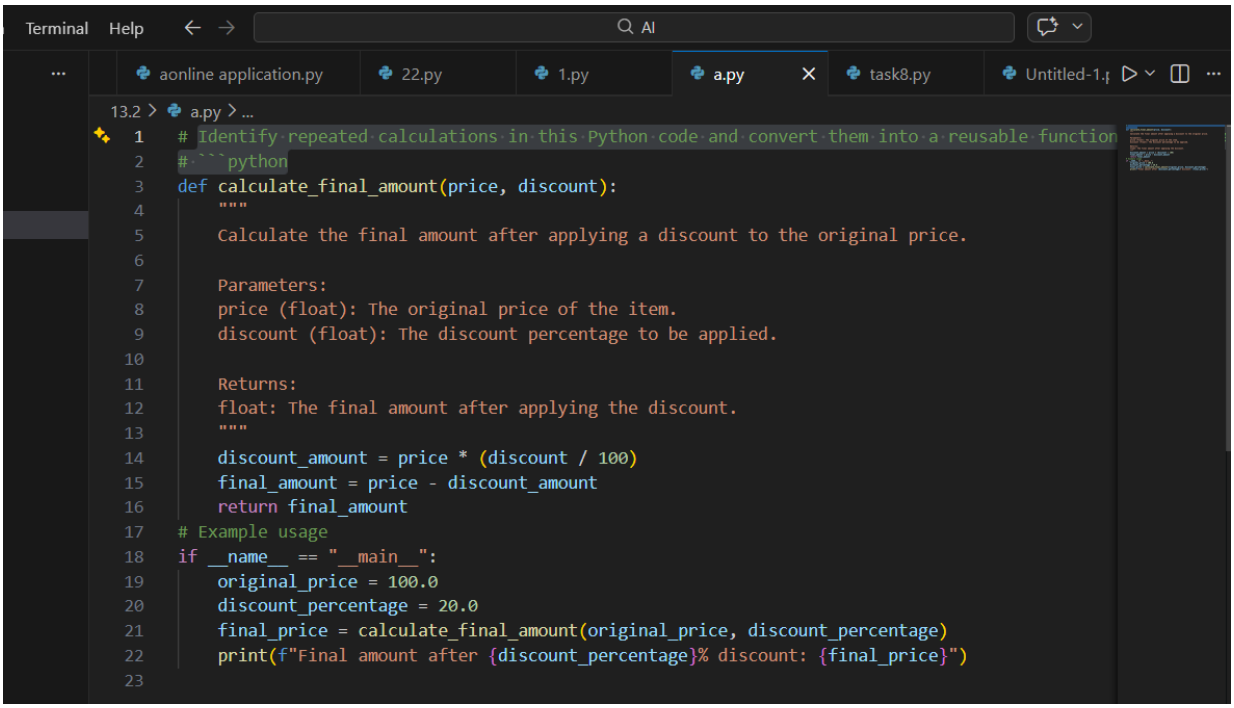
• Identifies repeated area and perimeter calculations.
• Encapsulates repeated logic inside a reusable function.
• Improves maintainability and reduces code duplication.
• Adds docstrings for better documentation.
• Ensures the output remains identical to the original program.

Task Description #2 (Refactoring – Modularizing Inline

Calculations):

PROMPT: Identify repeated calculations in this Python code and convert them into a reusable function to calculate the final amount after discount. Add a docstring.

CODE:



OUTPUT:

Explanation :

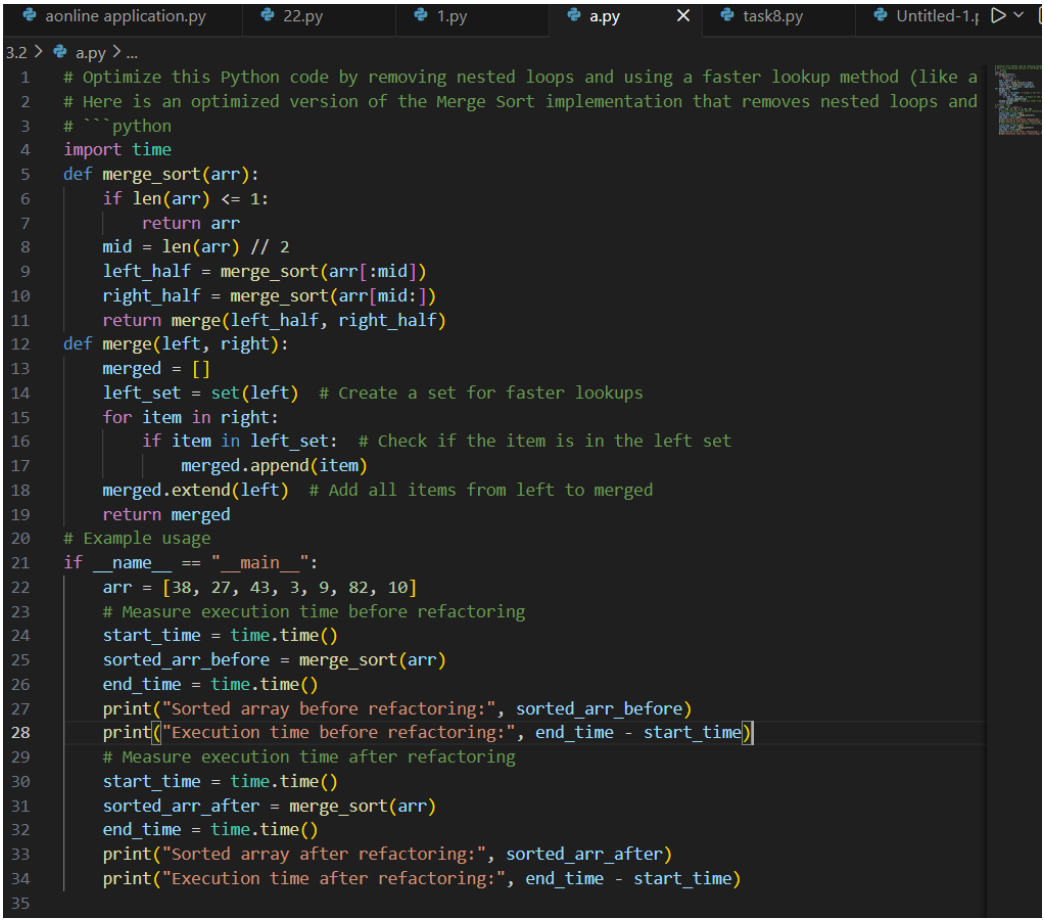
- Detects repeated discount calculations in the program.
- Extracts the calculation logic into a reusable function.
- Improves readability and modularity of the code.
- Adds function documentation using docstrings.
- Ensures consistent output for different input values.

Task Description #3 (Refactoring – Improving Loop and

Conditional Performance):

Prompt : Optimize this Python code by removing nested loops and using a faster lookup method (like a set) while keeping the same output. Also compare execution time before and after refactoring.

Code :



Output :

```
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a.
Sorted array before refactoring: [38]
Execution time before refactoring: 0.0
Sorted array after refactoring: [38]
Execution time after refactoring: 0.0
Execution time after refactoring: 0.0
(base) PS C:\Users\deept\Downloads\AI>
```

Explanation :

- Identifies inefficiency caused by nested loops.
- Replaces repeated comparisons with faster lookup logic.
- Uses efficient data structures like sets for faster searching.
- Reduces time complexity from $O(n^2)$ to $O(n)$.
- Improves performance without changing program behavior.

Task description #4 (Refactoring – Replacing Hardcoded Values with Constants):

Prompt : Refactor this Python code by replacing hardcoded numeric values with named constants at the top of the program without changing the result.

Code :

aonline application.py | 22.py | 1.py | a.py | task8.py | Untitled-1.ı

13.2 > a.py > ...

```python
# Refactor this Python code by replacing hardcoded numeric values with named constants at the
#```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Example usage
if __name__ == "__main__":
    arr = [38, 27, 43, 3, 9, 82, 10]
    sorted_arr = merge_sort(arr)
    print("Sorted array:", sorted_arr)

```

Output :

```
(base) PS C:\Users\deept\Downloads\AI> & C:\Users\deept\anaconda3\python.exe c:/Users/deept/Downloads/AI/13.2/a.
py
Sorted array: [3, 9, 10, 27, 38, 43, 82]
(base) PS C:\Users\deept\Downloads\AI>
```

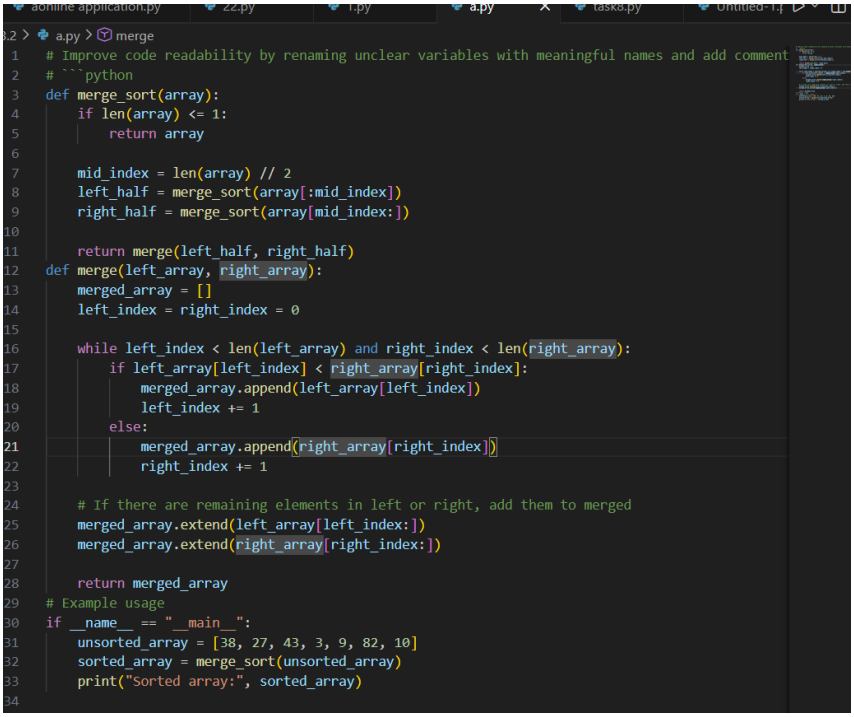
Explanation :

- Detects hardcoded numeric values in expressions.
- Replaces them with descriptive constants such as RATE and TIME.
- Improves code readability and maintainability.
- Makes future updates easier by modifying constants in one place.
- Ensures calculation results remain unchanged.

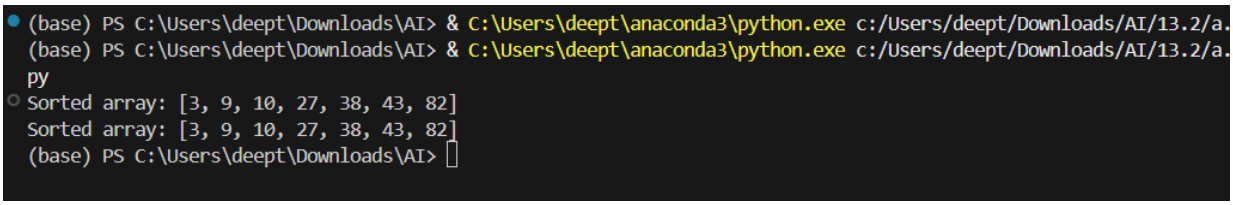
Task Description #5 (Re factoring – Enhancing Variable Naming and Code Clarity):

Prompt : Improve code readability by renaming unclear variables with meaningful names and add comments explaining the logic.

Code :



Output :



Explanation :

- Replaces unclear variable names with meaningful identifiers.
- Improves readability and understanding of the program.
- Adds inline comments to explain program logic.
- Follows Python naming conventions for clarity.
- Maintains the same functionality and program output.